rbhagava_Homework6

Rishi Shanthan Bhagavatham

2024-12-02

Chapter 9 R Commands

```
library(TSA)
```

```
##
## Attaching package: 'TSA'

## The following objects are masked from 'package:stats':
##
##     acf, arima

## The following object is masked from 'package:utils':
##
##     tar
```

Exhibit 9.2

```
data(tempdub)
tempdub1=ts(c(tempdub,rep(NA,24)),start=start(tempdub),freq=frequency(tempdub))
```

```
har.=harmonic(tempdub,1)
m5.tempdub=arima(tempdub,order=c(0,0,0),xreg=har.)
```

```
har.=harmonic(tempdub,1); model4=lm(tempdub~har.)
summary(model4)
```

```
##
## Call:
## lm(formula = tempdub ~ har.)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -11.1580  -2.2756  -0.1457   2.3754  11.2671
##
## Coefficients:
##                 Estimate Std. Error t value Pr(>|t|)
```

```
## (Intercept)      46.2660     0.3088 149.816  < 2e-16 ***
## har.cos(2*pi*t) -26.7079     0.4367 -61.154  < 2e-16 ***
## har.sin(2*pi*t)  -2.1697     0.4367  -4.968 1.93e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.706 on 141 degrees of freedom
## Multiple R-squared:  0.9639, Adjusted R-squared:  0.9634
## F-statistic:  1882 on 2 and 141 DF,  p-value: < 2.2e-16
```

```
arima(tempdub,order=c(0,0,0),xreg=data.frame(har.,trend=time(tempdub)))
```

```
##
## Call:
## arima(x = tempdub, order = c(0, 0, 0), xreg = data.frame(har., trend = time(tempdub)))
##
## Coefficients:
##       intercept  cos.2.pi.t.  sin.2.pi.t.   trend
##         23.8569     -26.7070      -2.1662  0.0114
## s.e.   174.1506       0.4322       0.4330  0.0884
##
## sigma^2 estimated as 13.45:  log likelihood = -391.43,  aic = 790.86
```

```
m5.tempdub
```

```
##
## Call:
## arima(x = tempdub, order = c(0, 0, 0), xreg = har.)
##
## Coefficients:
##       intercept  cos(2*pi*t)  sin(2*pi*t)
##         46.2660     -26.7079      -2.1697
## s.e.     0.3056       0.4322       0.4322
##
## sigma^2 estimated as 13.45:  log likelihood = -391.44,  aic = 788.88
```

```
newhar.=harmonic(ts(rep(1,24), start=c(1976,1),freq=12),1)
plot(m5.tempdub,n.ahead=24,n1=c(1972,1),newxreg=newhar., col = 'red', type='b',ylab='Temperature',
xlab='Year')
```

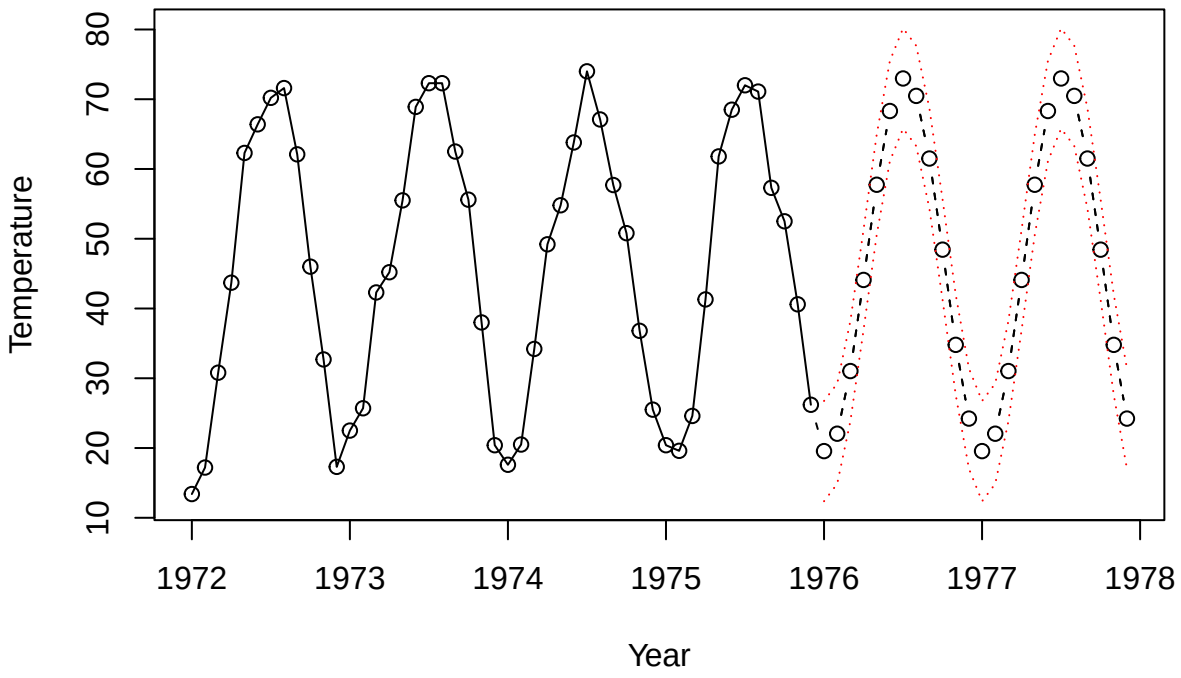
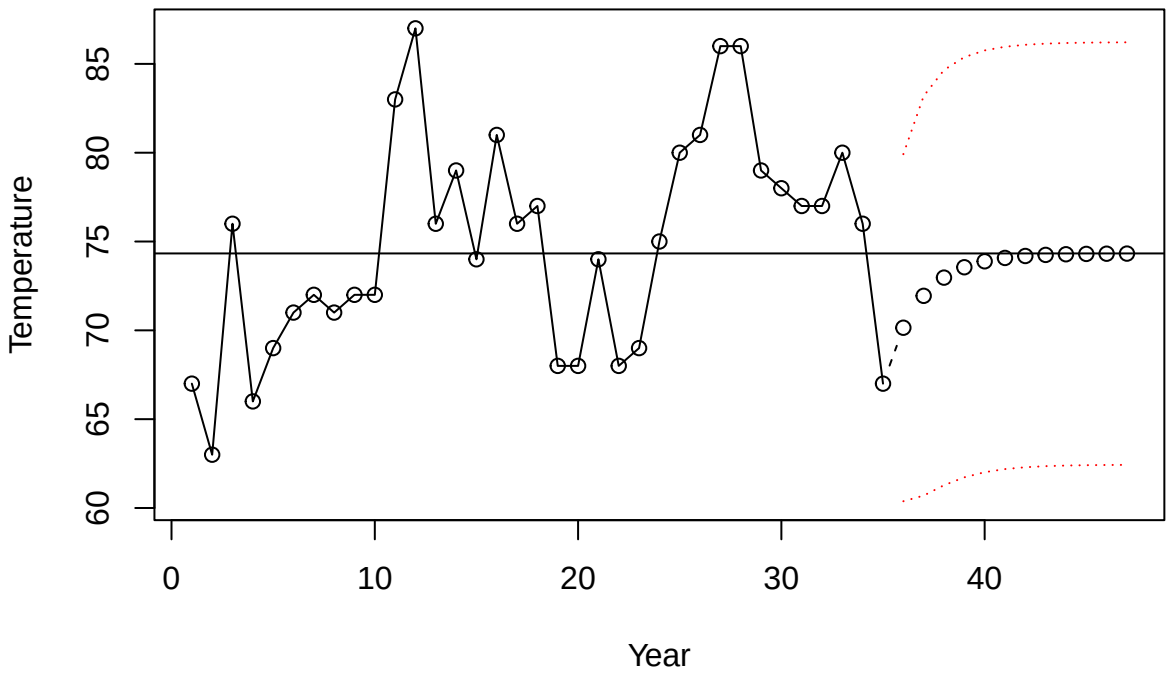


Exhibit 9.3

```
data(color)
m1.color=arima(color,order=c(1,0,0))
plot(m1.color,n.ahead=12,col='red',type='b',xlab='Year',ylab='Temperature')
abline(h=coef(m1.color)[names(coef(m1.color))=='intercept'])
```




```
v=1:5
names(v)
```

```
## NULL
```

```
names(v)=c('A','B','C','D','E')
names(v)=='C'
```

```
## [1] FALSE FALSE  TRUE FALSE FALSE
```

```
v[names(v)=='C']
```

```
## C
## 3
```

```
v[3]
```

```
## C
## 3
```

```
v[-3]
```

```
## A B D E
## 1 2 4 5
```

Chapter 10 R Commands

```
plot(y=ARMAacf(ma=c(0.5,rep(0,10),0.8,0.4),lag.max=13)[-1],x=1:13,type='h',xlab='Lag k',
ylab=expression(rho[k]),axes=F,ylim=c(0,0.6))
points(y=ARMAacf(ma=c(0.5,rep(0,10),0.8,0.4),lag.max=13)[-1],x=1:13,pch=20)
abline(h=0)
axis(1,at=1:13,
labels=c(1,NA,3,NA,5,NA,7,NA,9,NA,11,NA,13))
axis(2)
```

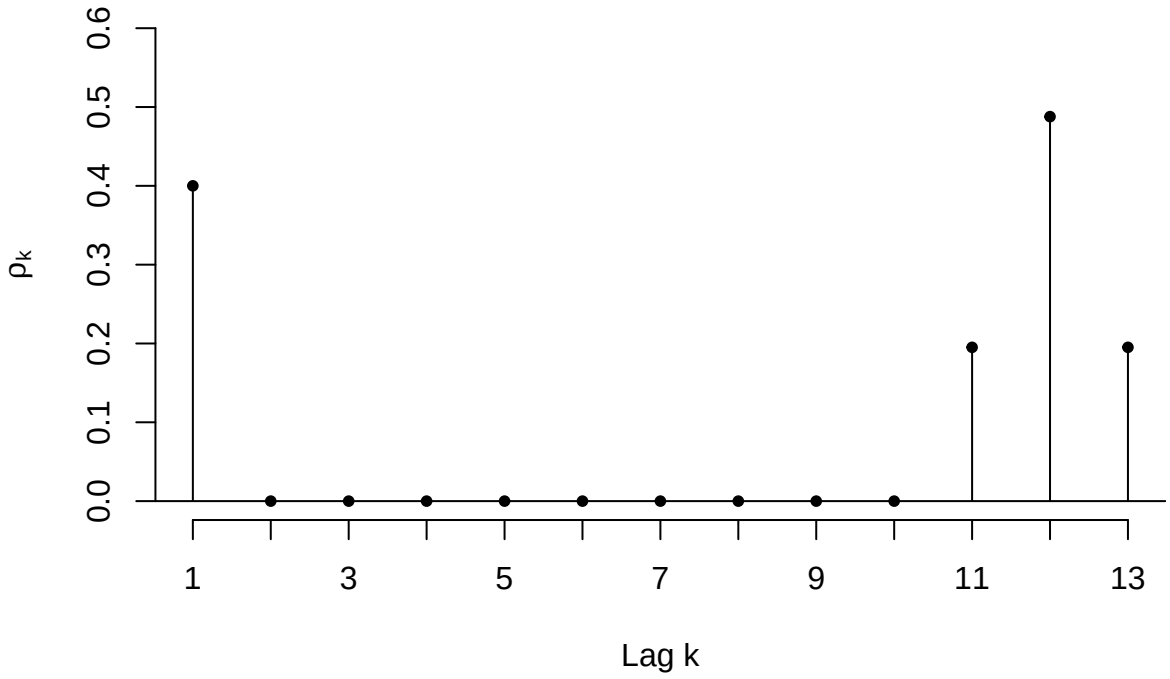


Exhibit 10.10

```
m1.co2=arima(co2,order=c(0,1,1),seasonal=list(order=c(0,1,1),period=12))
```

```
m1.co2
```

```
##
## Call:
## arima(x = co2, order = c(0, 1, 1), seasonal = list(order = c(0, 1, 1), period = 12))
##
## Coefficients:
##          ma1     sma1
```

```
##       -0.3501  -0.8506
## s.e.   0.0496   0.0257
##
## sigma^2 estimated as 0.0826:  log likelihood = -86.08,  aic = 176.16
```

Question 2

Exercise 9.9

```
set.seed(132456)
data_series = arima.sim(n = 48, list(ar = 0.8)) + 100
data_series
```

```
## Time Series:
## Start = 1
## End = 48
## Frequency = 1
##  [1]  99.32174 100.54041  99.18426 100.62163 100.70306 102.16264 101.18007
##  [8] 101.61750  99.17599 100.14493  99.77017  99.93022  99.42998 100.36642
## [15]  99.20188 101.42770 101.34206 101.54077 102.22879 102.08809 103.06755
## [22] 101.55893 101.39909 101.06198  99.14545  97.33168  97.70120  95.73272
## [29]  94.55397  96.66799  98.13306  99.88324 101.57048 102.75689 101.81881
## [36] 102.22101 100.98817 100.24840  98.71716  97.60278  98.08208  97.23044
## [43]  99.10154  99.92605  99.91232 100.52927 100.77788  99.19141
```

```
future_values = window(data_series, start = 41)
data_series = window(data_series, end = 40)
future_values
```

```
## Time Series:
## Start = 41
## End = 48
## Frequency = 1
## [1]  98.08208  97.23044  99.10154  99.92605  99.91232 100.52927 100.77788
## [8]  99.19141
```

a

```
arima_model = arima(data_series, order = c(1, 0, 0))
arima_model
```

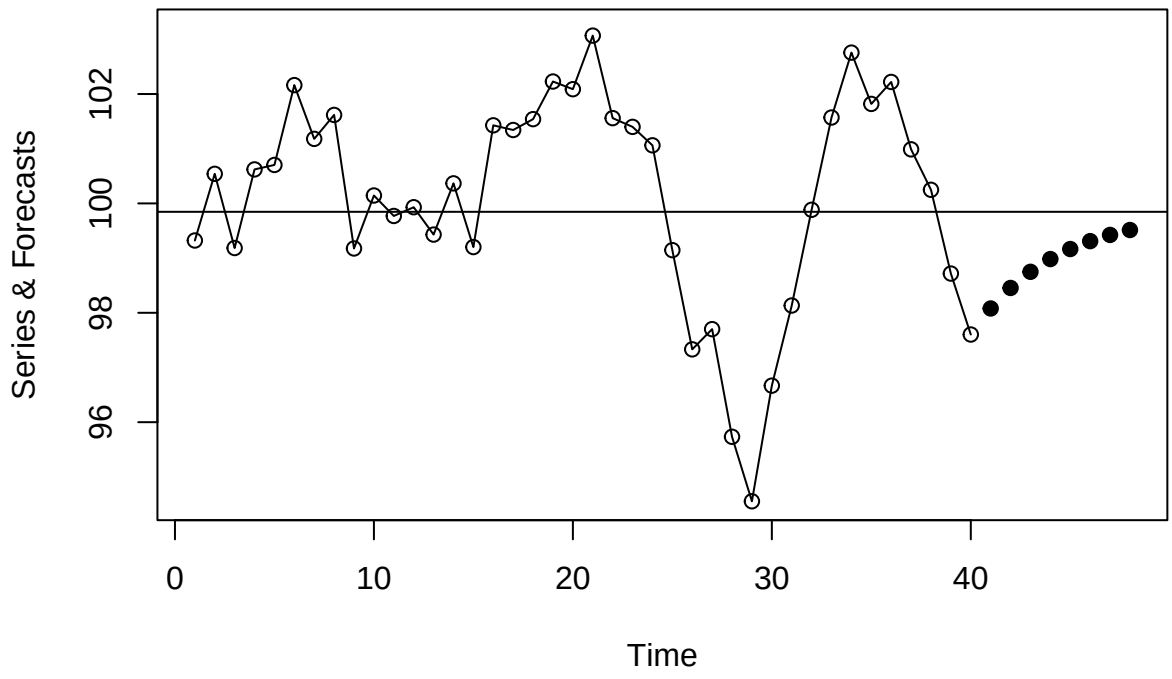
```
##
## Call:
## arima(x = data_series, order = c(1, 0, 0))
##
## Coefficients:
##          ar1  intercept
##       0.7878    99.8465
```

```
## s.e.  0.0943     0.8110
##
## sigma^2 estimated as 1.372:  log likelihood = -63.57,  aic = 131.14
```

The maximum likelihood estimates are highly accurate in this specific simulation.

b

```
plot(arima_model, n.ahead = 8, ylab = 'Series & Forecasts', col = NULL, pch = 19)
abline(h = coef(arima_model)[names(coef(arima_model)) == 'intercept'])
```



c

```
forecasted_values = predict(arima_model, n.ahead = 8)$pred
comparison_df = data.frame(Actual = future_values, Forecast = forecasted_values)
print(comparison_df)
```

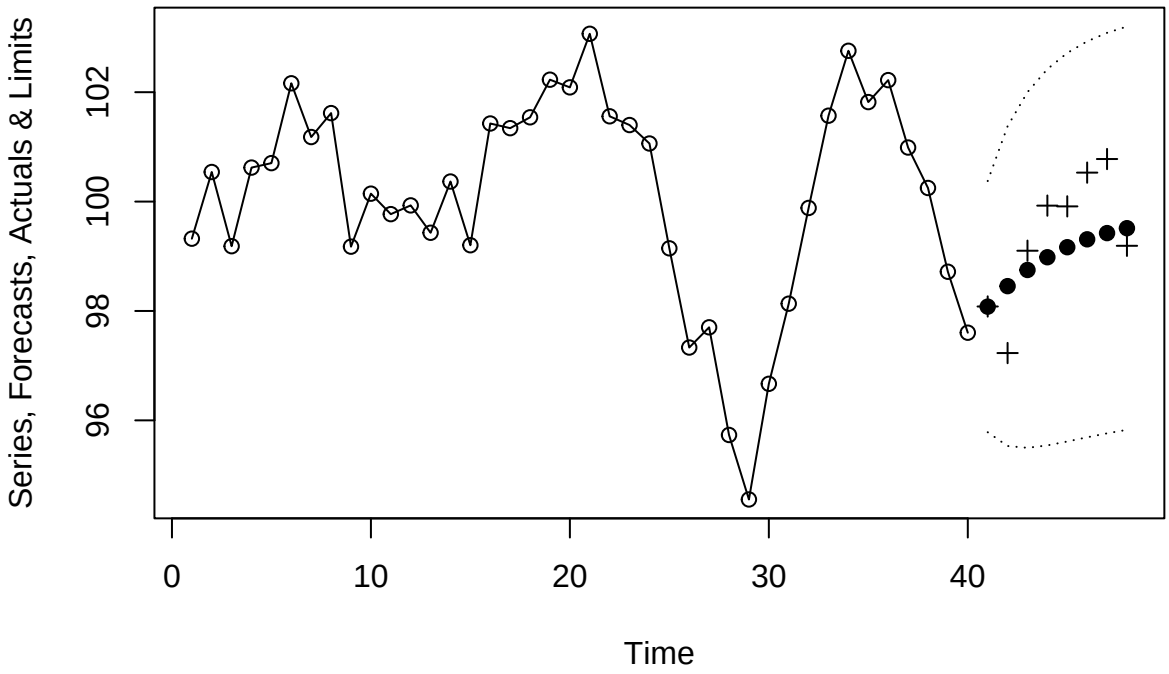
```
##      Actual Forecast
## 1  98.08208 98.07883
## 2  97.23044 98.45387
## 3  99.10154 98.74934
```

```
## 4  99.92605 98.98213
## 5  99.91232 99.16553
## 6 100.52927 99.31001
## 7 100.77788 99.42384
## 8  99.19141 99.51353
```

```
forecast_error = future_values - forecasted_values
cat("Forecast errors: ", forecast_error, "\n")
```

```
## Forecast errors:  0.003249388 -1.223434 0.3521927 0.9439243 0.7467971 1.21926 1.354032 -0.3221114
```

```
plot(arima_model, n.ahead = 8, ylab = 'Series, Forecasts, Actuals & Limits', pch = 19)
points(x = (41:48), y = future_values, pch = 3)
```

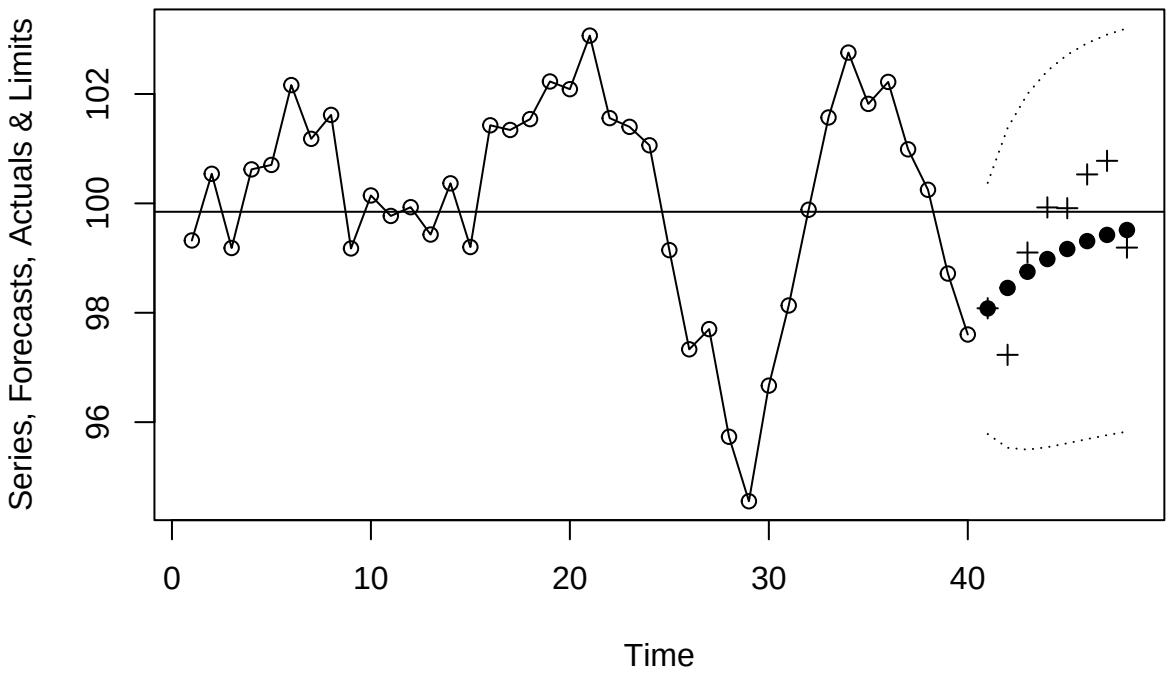


The actual future values of the series are represented by plus signs (+) on the plot.

The forecast errors are minimal, indicating that the AR(1) model has provided accurate predictions, with the errors close to zero. The forecasts are generally within the 95% confidence limits, demonstrating strong predictive performance.

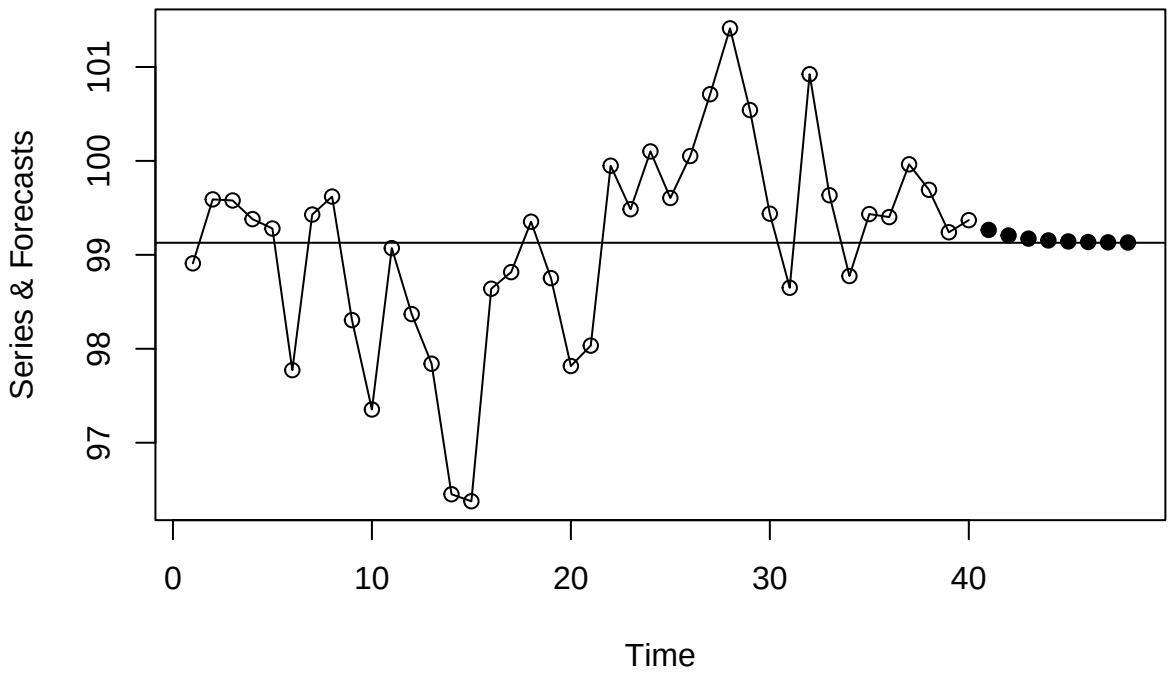
d

```
plot(arima_model, n.ahead = 8, ylab = 'Series, Forecasts, Actuals & Limits', pch = 19)
points(x = (41:48), y = future_values, pch = 3)
abline(h = coef(arima_model)[names(coef(arima_model)) == 'intercept'])
```

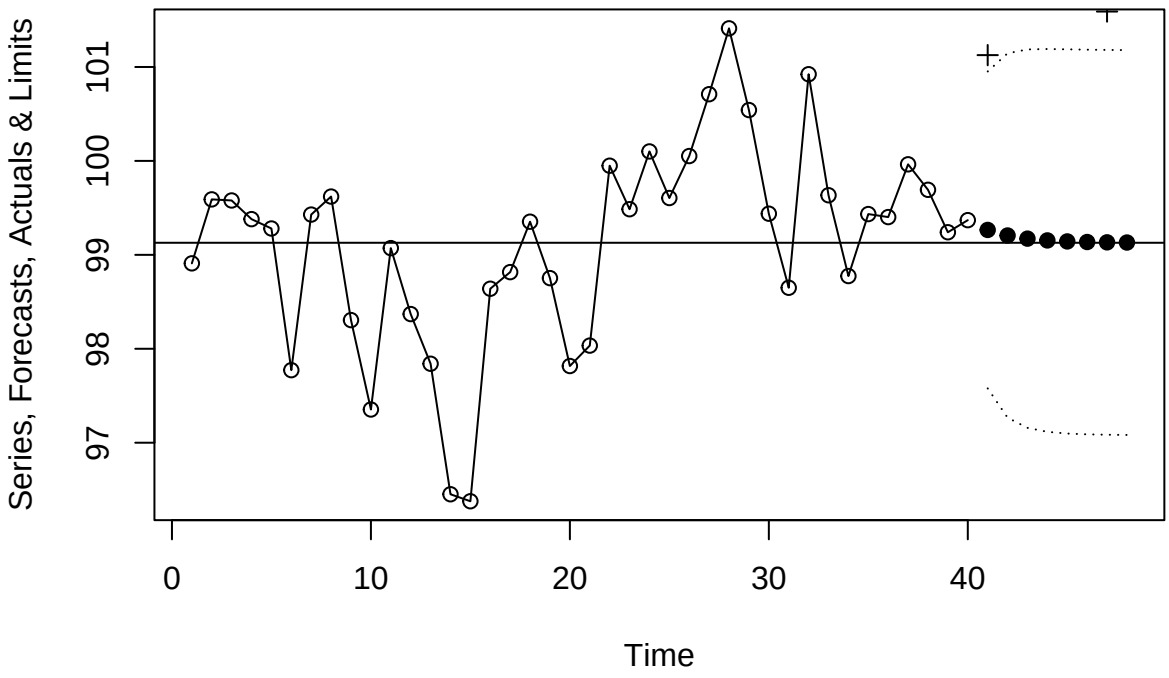


e

```
set.seed(98765)
data_series = arima.sim(n = 48, list(ar = 0.8)) + 100
future_values = window(data_series, start = 41)
data_series = window(data_series, end = 40)
arima_model = arima(data_series, order = c(1, 0, 0))
plot(arima_model, n.ahead = 8, ylab = 'Series & Forecasts', col = NULL, pch = 19)
abline(h = coef(arima_model)[names(coef(arima_model)) == 'intercept'])
```




```
plot(arima_model, n.ahead = 8, ylab = 'Series, Forecasts, Actuals & Limits', pch = 19)
points(x = (41:48), y = future_values, pch = 3)
abline(h = coef(arima_model)[names(coef(arima_model)) == 'intercept'])
```



Exercise 9.12

```
set.seed(123)
data_series = arima.sim(n=36, list(ma=c(-1, 0.6))) + 100
test_data = window(data_series, start=33)
data_series = window(data_series, end=32)
```

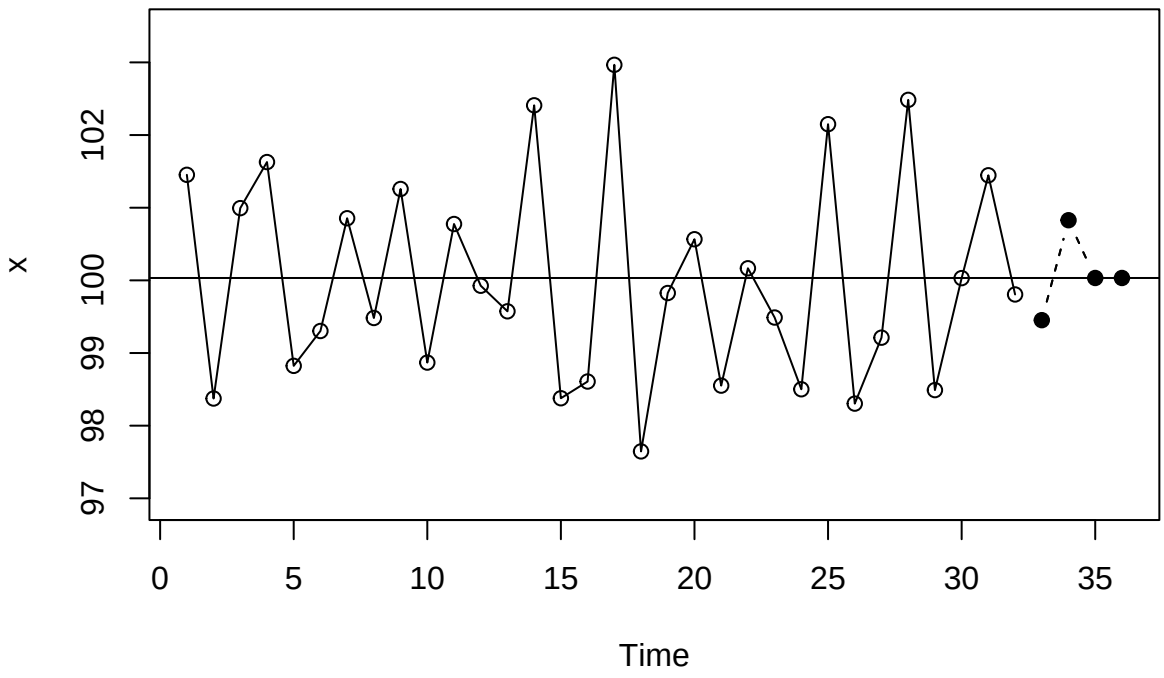
a

```
model_fit = arima(data_series, order=c(0, 0, 2))
model_fit
```

```
##
## Call:
## arima(x = data_series, order = c(0, 0, 2))
##
## Coefficients:
##           ma1     ma2  intercept
##       -1.2776  1.0000   100.0326
## s.e.   0.1403  0.1641     0.1023
##
## sigma^2 estimated as 0.6759:  log likelihood = -42.23,  aic = 90.47
```

b

```
forecast_graph = plot(model_fit, n.ahead=4, col=NULL, pch=19)
abline(h=coef(model_fit)[names(coef(model_fit)) == "intercept"])
```



c

For the MA(2) model, the forecasts at lead times 3 and 4 are essentially the estimated process mean, which is why the data points align with the mean line.

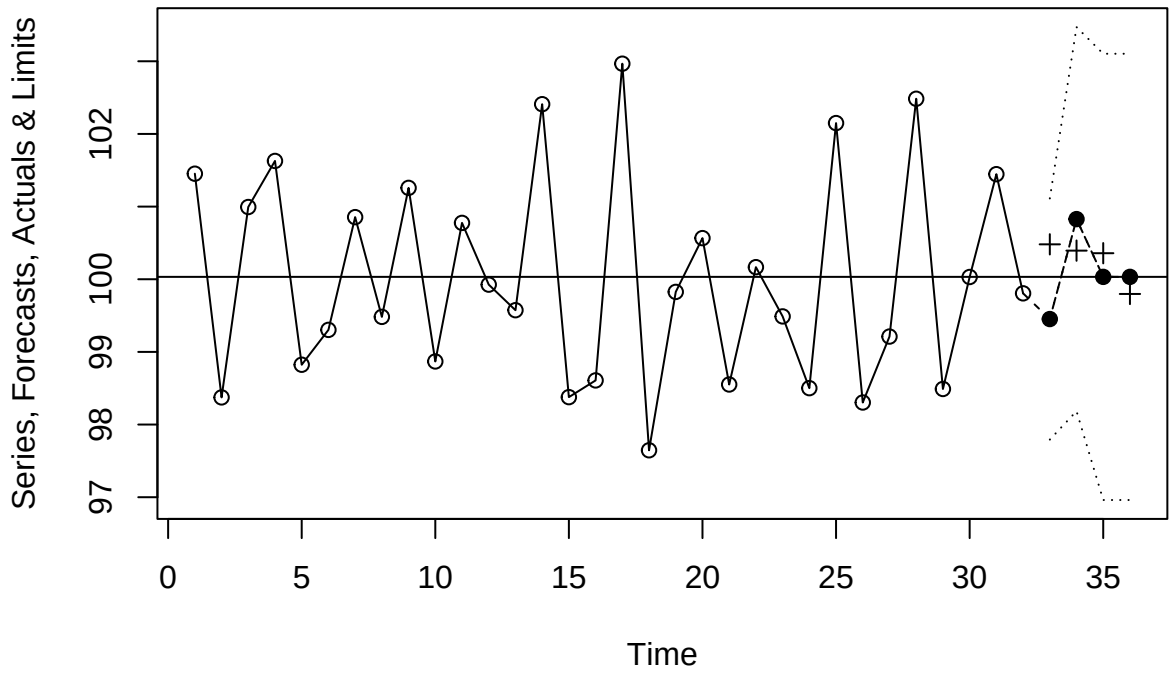
d

```
cbind(test_data, forecast_graph$pred)
```

```
## Time Series:
## Start = 33
## End = 36
## Frequency = 1
##    test_data forecast_graph$pred
## 33 100.48052              99.4522
## 34 100.39394             100.8275
## 35 100.35823             100.0326
## 36  99.79735             100.0326
```

e

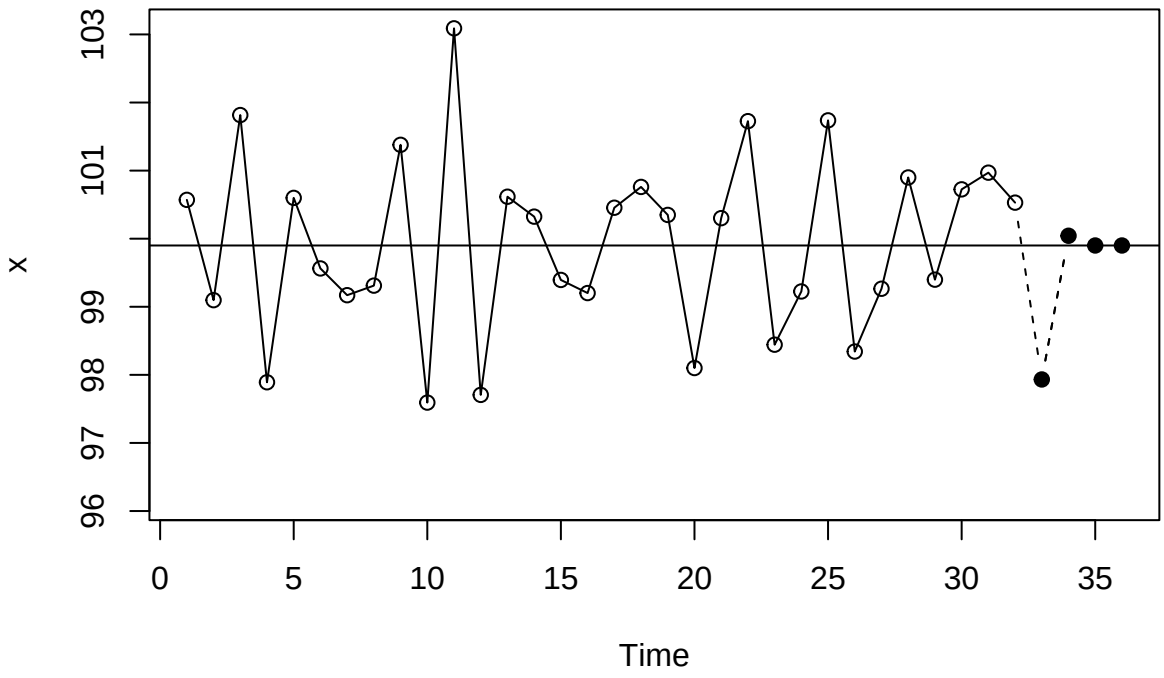
```
plot(model_fit, n.ahead=4, ylab="Series, Forecasts, Actuals & Limits", type='o', pch=19)
points(x=(33:36), y=test_data, pch=3)
abline(h=coef(model_fit)[names(coef(model_fit)) == "intercept"])
```



All actual values fall within the forecast confidence intervals.

f

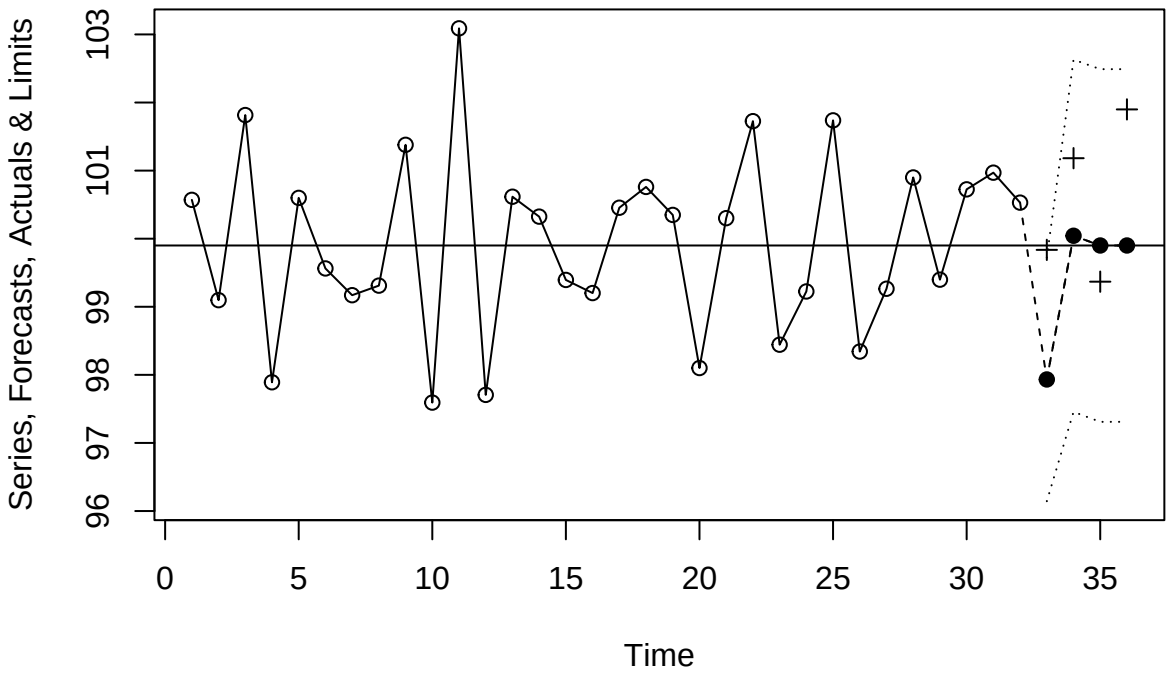
```
set.seed(987)
data_series = arima.sim(n=36, list(ma=c(-1, 0.6))) + 100
test_data = window(data_series, start=33)
data_series = window(data_series, end=32)
model_fit = arima(data_series, order=c(0, 0, 2))
forecast_graph = plot(model_fit, n.ahead=4, col=NULL, pch=19)
abline(h=coef(model_fit)[names(coef(model_fit)) == "intercept"])
```




```
cbind(test_data, forecast_graph$pred)
```

```
## Time Series:
## Start = 33
## End = 36
## Frequency = 1
##    test_data forecast_graph$pred
## 33  99.83595            97.93229
## 34 101.18218           100.04310
## 35  99.36855            99.89990
## 36 101.89890            99.89990
```

```
plot(model_fit, n.ahead=4, ylab="Series, Forecasts, Actuals & Limits", type='o', pch=19)
points(x=(33:36), y=test_data, pch=3)
abline(h=coef(model_fit)[names(coef(model_fit)) == "intercept"])
```



In this particular simulation and model, the forecasts deviate significantly from the actual values.

Exercise 9.13

```
set.seed(5323)
data_series = arima.sim(n=50, list(ar=0.7, ma=-0.5)) + 100
actual_values = window(data_series, start=41)
data_series_to_40 = window(data_series, end=40)
```

a

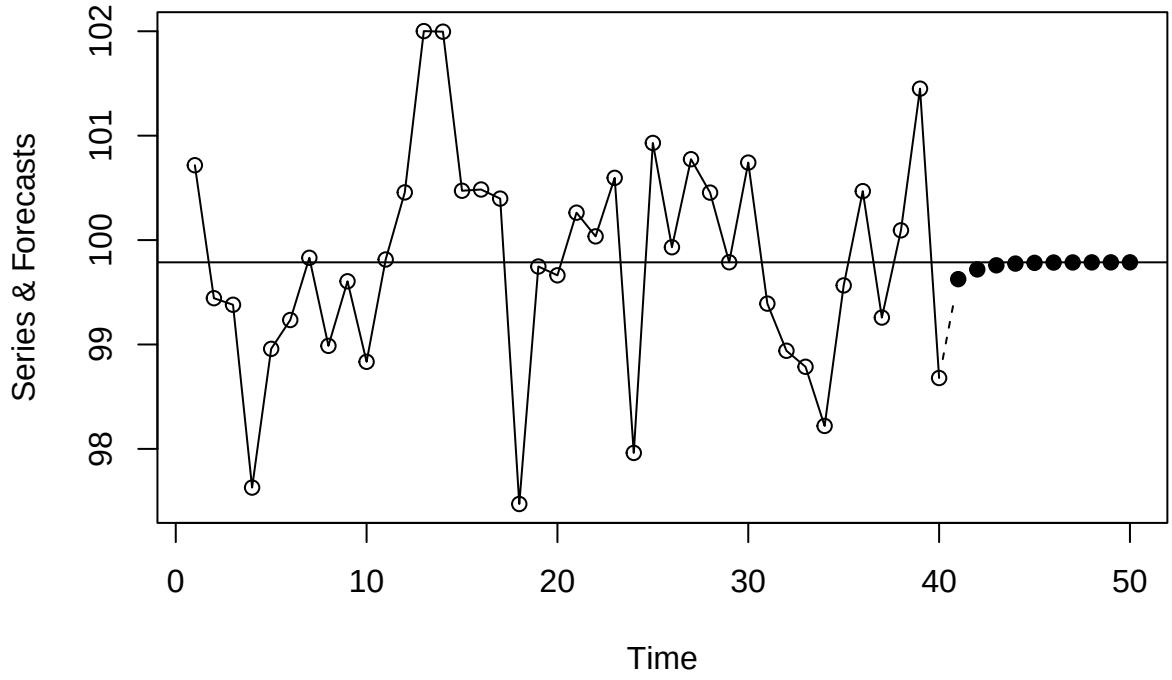
```
model_fit = arima(data_series_to_40, order=c(1, 0, 1))
model_fit
```

```
##
## Call:
## arima(x = data_series_to_40, order = c(1, 0, 1))
##
## Coefficients:
##          ar1      ma1  intercept
##       0.4212  -0.2074    99.7869
## s.e.  0.3562   0.3679     0.2145
```

```
##
## sigma^2 estimated as 1.004:  log likelihood = -56.87,  aic = 119.73
```

b

```
forecast_plot = plot(model_fit, n.ahead=10, ylab="Series & Forecasts", col=NULL, pch=19)
abline(h=coef(model_fit)[names(coef(model_fit)) == "intercept"])
```



The forecasts converge to the series mean relatively quickly.

c

```
comparison = cbind(actual_values, forecast_plot$pred)
comparison
```

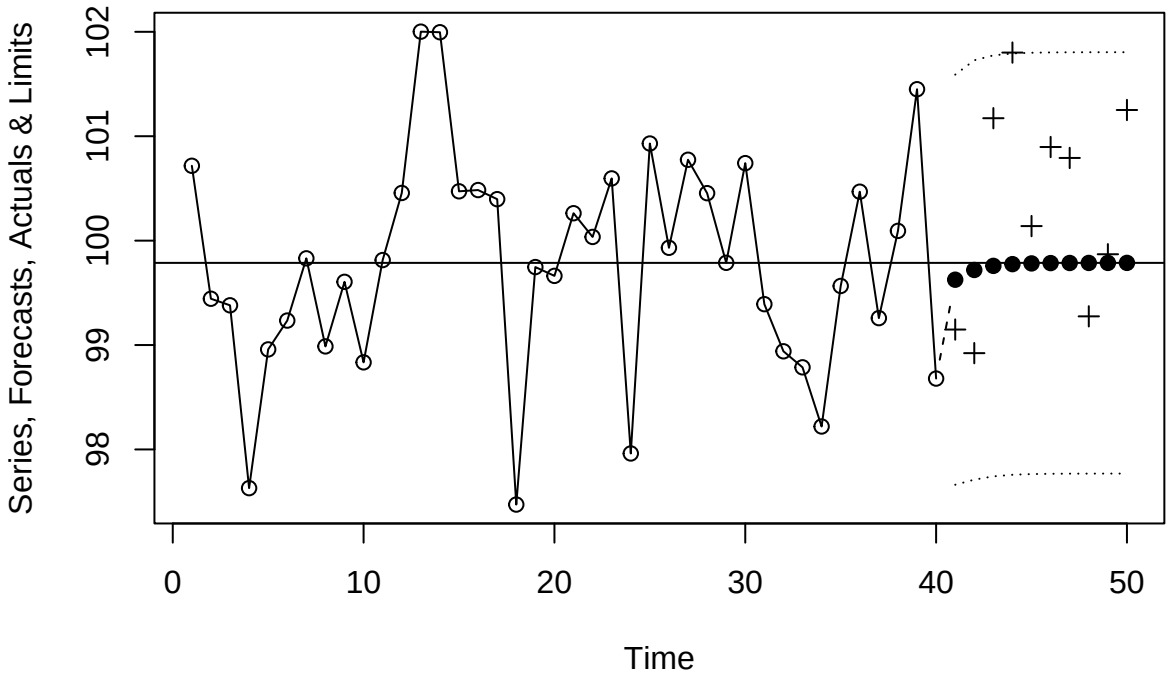
```
## Time Series:
## Start = 41
## End = 50
## Frequency = 1
##    actual_values forecast_plot$pred
## 41      99.14840           99.62577
## 42      98.92234           99.71901
```

```
## 43     101.17251          99.75828
## 44     101.80098          99.77482
## 45     100.13930          99.78179
## 46     100.89658          99.78472
## 47     100.79189          99.78596
## 48      99.27489          99.78648
## 49      99.86969          99.78670
## 50     101.25117          99.78679
```

The predicted values are very similar to the actual ones.

d

```
forecast_limit_plot = plot(model_fit, n.ahead=10, ylab="Series, Forecasts, Actuals & Limits", pch=19)
points(x=41:50, y=actual_values, pch=3)
abline(h=coef(model_fit)[names(coef(model_fit)) == "intercept"])
```

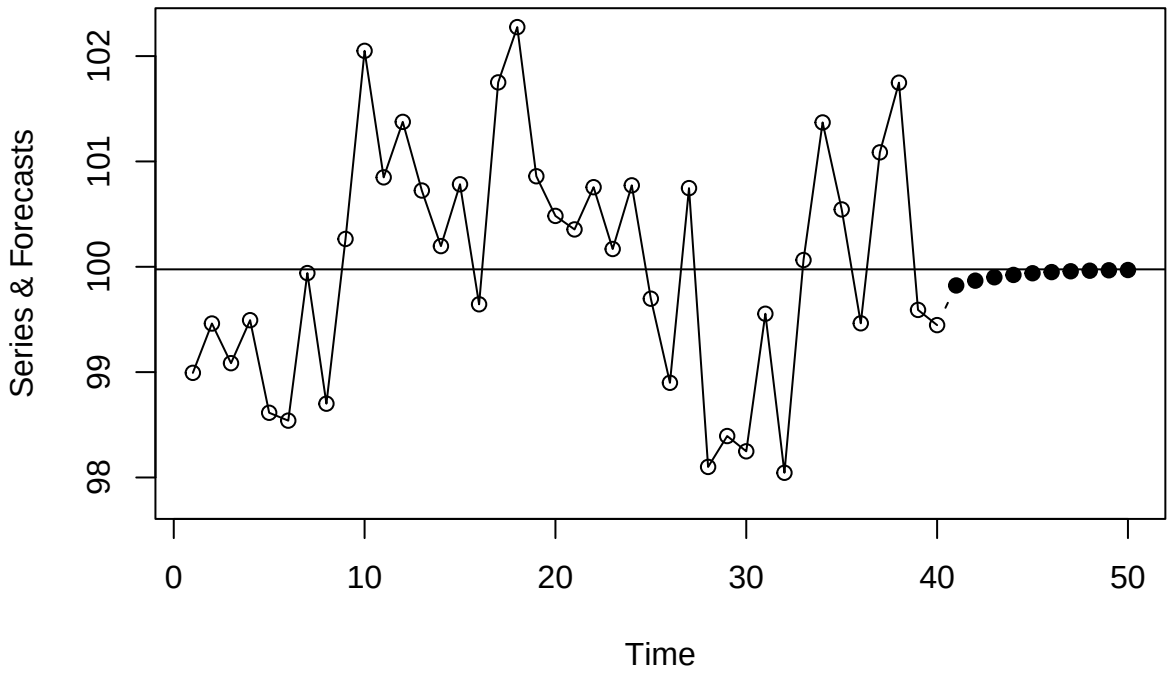


The actual values fall within the forecast limits, but the forecasts rapidly approach the estimated process mean, and the prediction intervals are relatively broad.

e

```r
set.seed(2353)
new_data_series = arima.sim(n=50, list(ar=0.7, ma=-0.5)) + 100
new_actual_values = window(new_data_series, start=41)
new_data_series_to_40 = window(new_data_series, end=40)

new_model_fit = arima(new_data_series_to_40, order=c(1, 0, 1))
new_forecast_plot = plot(new_model_fit, n.ahead=10, ylab="Series & Forecasts", col=NULL, pch=19)
abline(h=coef(new_model_fit)[names(coef(new_model_fit)) == "intercept"])
```

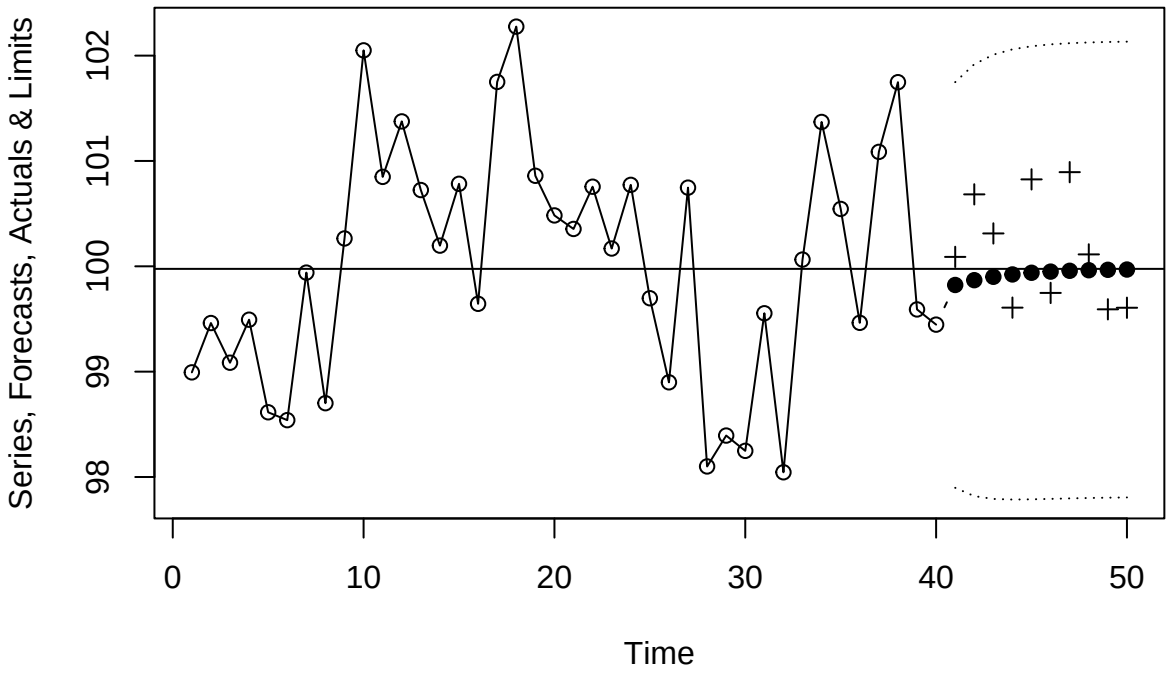



```r
new_comparison = cbind(new_actual_values, new_forecast_plot$pred)
new_comparison
```

```
## Time Series:
## Start = 41
## End = 50
## Frequency = 1
##    new_actual_values new_forecast_plot$pred
## 41         100.08924                99.82260
## 42         100.68292                99.86825
## 43         100.31205                99.90028
## 44          99.60758                99.92276
## 45         100.82513                99.93853
## 46          99.74718                99.94959
## 47         100.89360                99.95735
```

```
## 48         100.11310              99.96280
## 49          99.59170              99.96662
## 50          99.60645              99.96930
```

```
new_forecast_limit_plot = plot(new_model_fit, n.ahead=10, ylab="Series, Forecasts, Actuals & Limits", p
points(x=41:50, y=new_actual_values, pch=3)
abline(h=coef(new_model_fit)[names(coef(new_model_fit)) == "intercept"])
```



Exercise 9.16

```
set.seed(8899)
time_series = arima.sim(n=45, list(order=c(0, 2, 2), ma=c(-1, 0.75)))
time_series = (time_series[-1])[-1]
actual_values = window(time_series, start=41)
time_series_upto_40 = window(time_series, end=40)
time_series
```

```
##  [1]   -0.4451137   -1.4091980   -3.4278513   -6.6341654   -6.3568065
##  [6]   -8.3667047  -11.4893176  -12.5944253  -13.5215139  -14.6795142
## [11]  -16.9326527  -17.7348829  -18.9447112  -20.5897099  -21.6184994
## [16]  -23.0420777  -25.0185752  -26.1402509  -29.5212823  -31.2011344
## [21]  -33.1955634  -35.1118104  -36.6386615  -38.8893063  -39.3983149
## [26]  -39.8087391  -41.5074975  -41.7382600  -44.6557347  -47.2895735
```

```
## [31]  -50.4134054  -54.9727334  -58.0817164  -64.2203362  -68.8251605
## [36]  -73.0989777  -77.4711434  -81.1571110  -84.4664929  -89.3466802
## [41]  -93.0098986  -96.5500250 -100.5211211 -104.5557101 -107.1139236
```

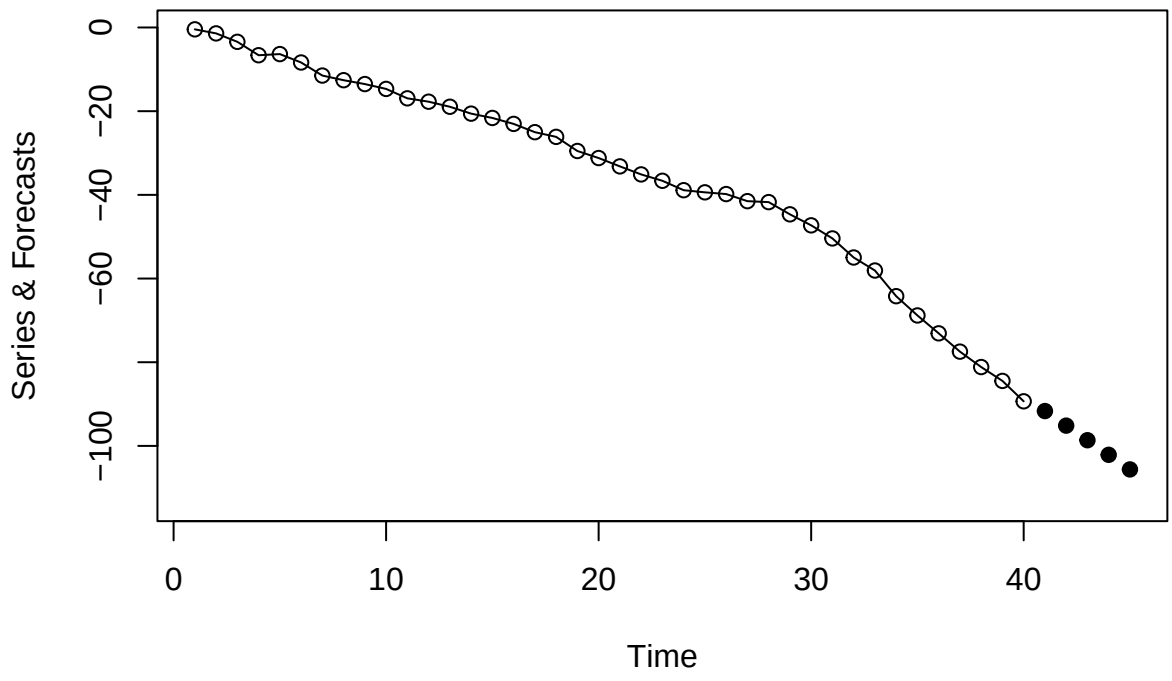
a

```
model_fit = arima(time_series_upto_40, order=c(0, 2, 2))
model_fit
```

```
##
## Call:
## arima(x = time_series_upto_40, order = c(0, 2, 2))
##
## Coefficients:
##           ma1     ma2
##       -1.2654  1.0000
## s.e.   0.1037  0.1357
##
## sigma^2 estimated as 0.8906:  log likelihood = -54.97,  aic = 113.94
```

b

```
forecast_result = plot(model_fit, n.ahead=5, ylab="Series & Forecasts", col=NULL, pch=19)
```



The forecasts appear to form a straight line.

c

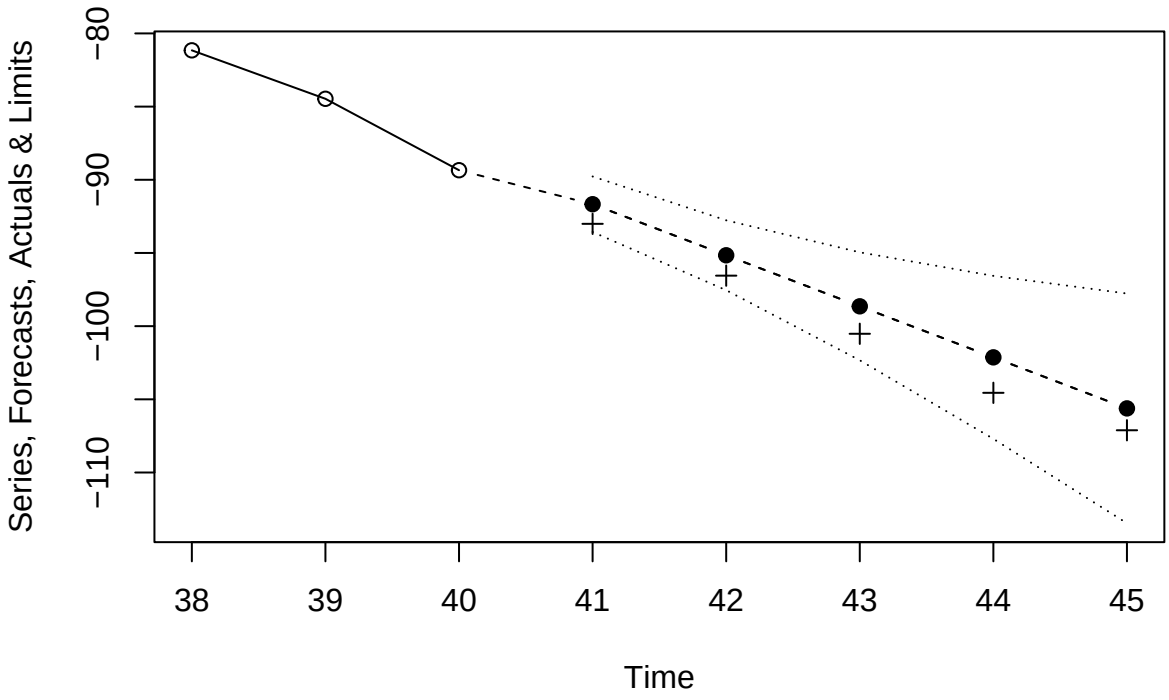
```
forecast_values = forecast_result$pred
comparison = cbind(actual_values, forecast_values)
comparison
```

```
## Time Series:
## Start = 41
## End = 45
## Frequency = 1
##    actual_values forecast_values
## 41     -93.00990       -91.66983
## 42     -96.55003       -95.15725
## 43    -100.52112       -98.64467
## 44    -104.55571      -102.13209
## 45    -107.11392      -105.61951
```

All forecasts are slightly higher than the actual values.

d

```
forecast_with_limits = plot(model_fit, n1=38, n.ahead=5, ylab="Series, Forecasts, Actuals & Limits", pc
points(x=seq(41, 45), y=actual_values, pch=3)
```



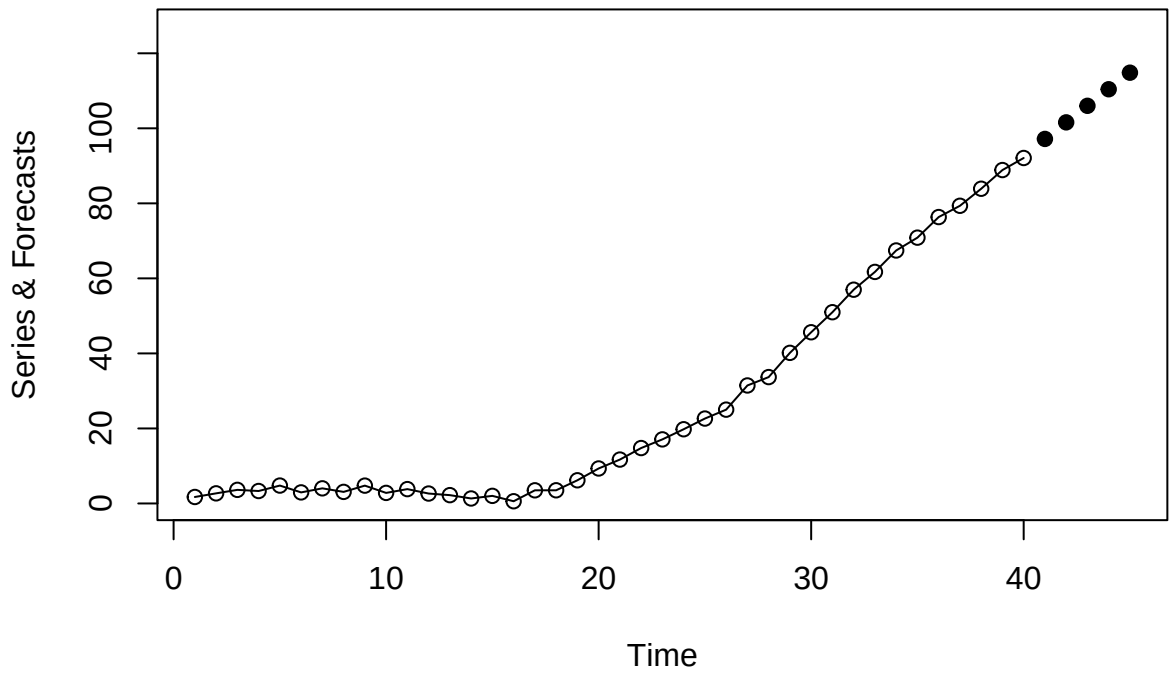
The actual values lie within the forecast limits. As the lead time increases, the forecast limits expand, which is characteristic of nonstationary models. Notably, the forecast for lead time 1 is very close to the lower limit.

e

```
set.seed(9876)
new_time_series = arima.sim(n=45, list(order=c(0, 2, 2), ma=c(-1, 0.75)))
new_time_series = (new_time_series[-1])[-1]
new_actual_values = window(new_time_series, start=41)
new_time_series_upto_40 = window(new_time_series, end=40)
new_time_series
```

```
##  [1]   1.7316181   2.6944340   3.6259939   3.3200515   4.7728511   2.9528472
##  [7]   4.0340646   3.0900245   4.7555550   2.8175124   3.8304772   2.6471112
## [13]   2.2231767   1.3356895   2.0304483   0.5953426   3.5052157   3.5142463
## [19]   6.1782882   9.3279490  11.6999732  14.7715973  17.0841475  19.7829442
## [25]  22.6331389  25.0117129  31.4537359  33.6902474  40.1479809  45.6399023
## [31]  50.9812665  56.9795691  61.7257705  67.4420304  70.8551539  76.3483011
## [37]  79.3536521  83.8963268  88.8794644  92.0925672  97.9872728 103.0996905
## [43] 106.9582046 113.1669728 116.8845109
```

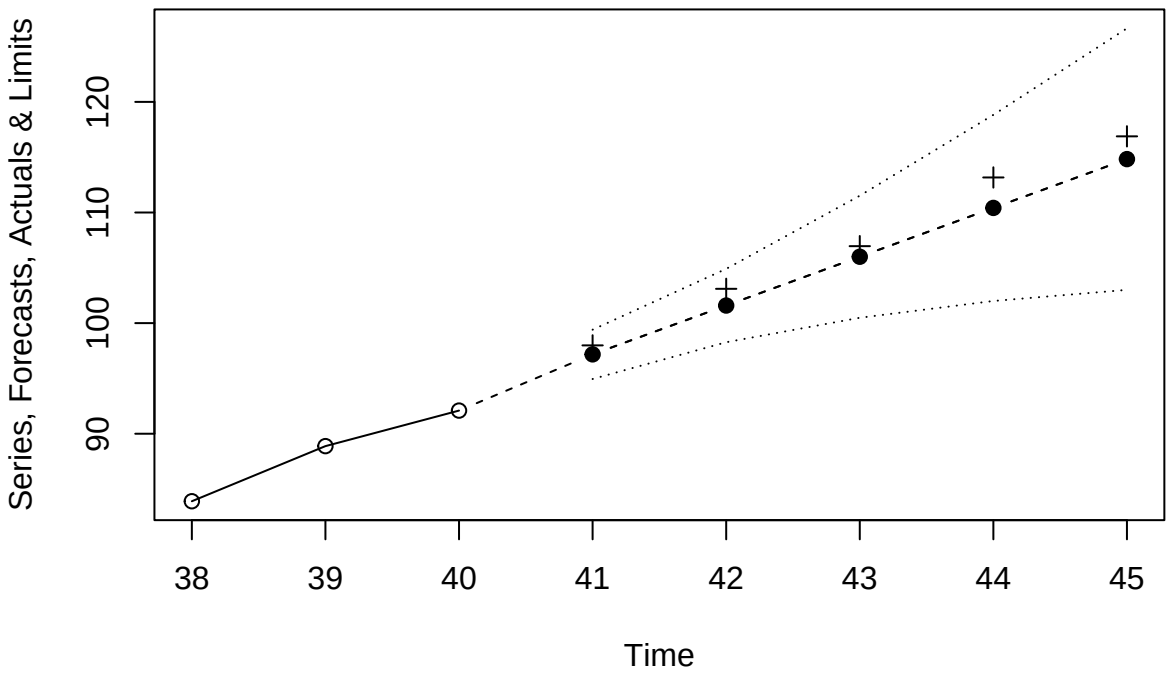
```
new_model_fit = arima(new_time_series_upto_40, order=c(0, 2, 2))
new_forecast_result = plot(new_model_fit, n.ahead=5, ylab="Series & Forecasts", col=NULL, pch=19)
```




```
new_forecast_values = new_forecast_result$pred
new_comparison = cbind(new_actual_values, new_forecast_values)
new_comparison
```

```
## Time Series:
## Start = 41
## End = 45
## Frequency = 1
##    new_actual_values new_forecast_values
## 41          97.98727            97.17485
## 42         103.09969           101.58773
## 43         106.95820           106.00062
## 44         113.16697           110.41350
## 45         116.88451           114.82638
```

```
new_forecast_with_limits = plot(new_model_fit, n1=38, n.ahead=5, ylab="Series, Forecasts, Actuals & Lim
points(x=seq(41, 45), y=new_actual_values, pch=3)
```



Exercise 10.8

a

```
data(co2)
seasonal_component = season(co2)
time_trend = time(co2)
trend_model = lm(co2 ~ seasonal_component + time_trend)
summary(trend_model)
```

```
##
## Call:
## lm(formula = co2 ~ seasonal_component + time_trend)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.73874 -0.59689 -0.06947  0.54086  2.15539
##
## Coefficients:
##                              Estimate Std. Error t value Pr(>|t|)
## (Intercept)                -3290.5412    44.1790 -74.482  < 2e-16 ***
## seasonal_componentFebruary     0.6682     0.3424   1.952 0.053320 .
## seasonal_componentMarch        0.9637     0.3424   2.815 0.005715 **
```

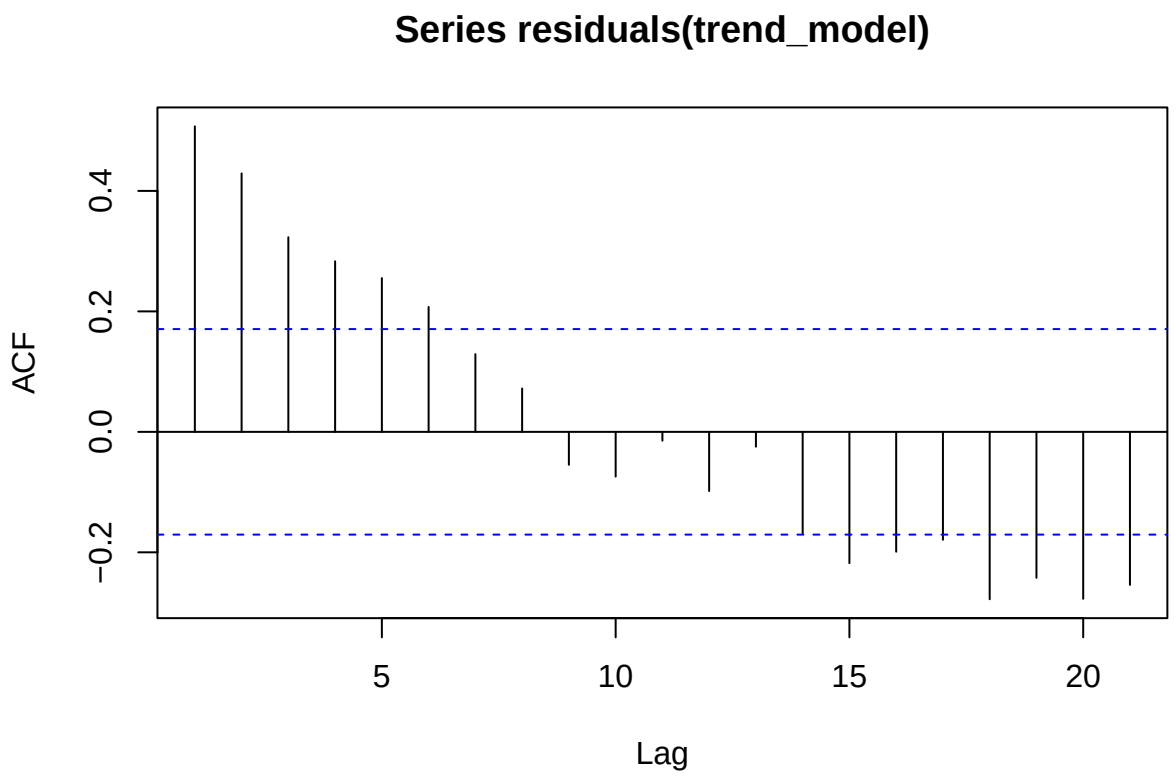
```
## seasonal_componentApril         1.2311     0.3424   3.595 0.000473 ***
## seasonal_componentMay           1.5275     0.3424   4.460 1.87e-05 ***
## seasonal_componentJune         -0.6761     0.3425  -1.974 0.050696 .
## seasonal_componentJuly         -7.2851     0.3426 -21.267  < 2e-16 ***
## seasonal_componentAugust      -13.4414     0.3426 -39.232  < 2e-16 ***
## seasonal_componentSeptember   -12.8205     0.3427 -37.411  < 2e-16 ***
## seasonal_componentOctober      -8.2604     0.3428 -24.099  < 2e-16 ***
## seasonal_componentNovember     -3.9277     0.3429 -11.455  < 2e-16 ***
## seasonal_componentDecember     -1.3367     0.3430  -3.897 0.000161 ***
## time_trend                      1.8321     0.0221  82.899  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.8029 on 119 degrees of freedom
## Multiple R-squared:  0.9902, Adjusted R-squared:  0.9892
## F-statistic: 997.7 on 12 and 119 DF,  p-value: < 2.2e-16
```

b

Multiple R-squared: 0.9902

c

```
residual_autocorrelation = acf(residuals(trend_model))
```



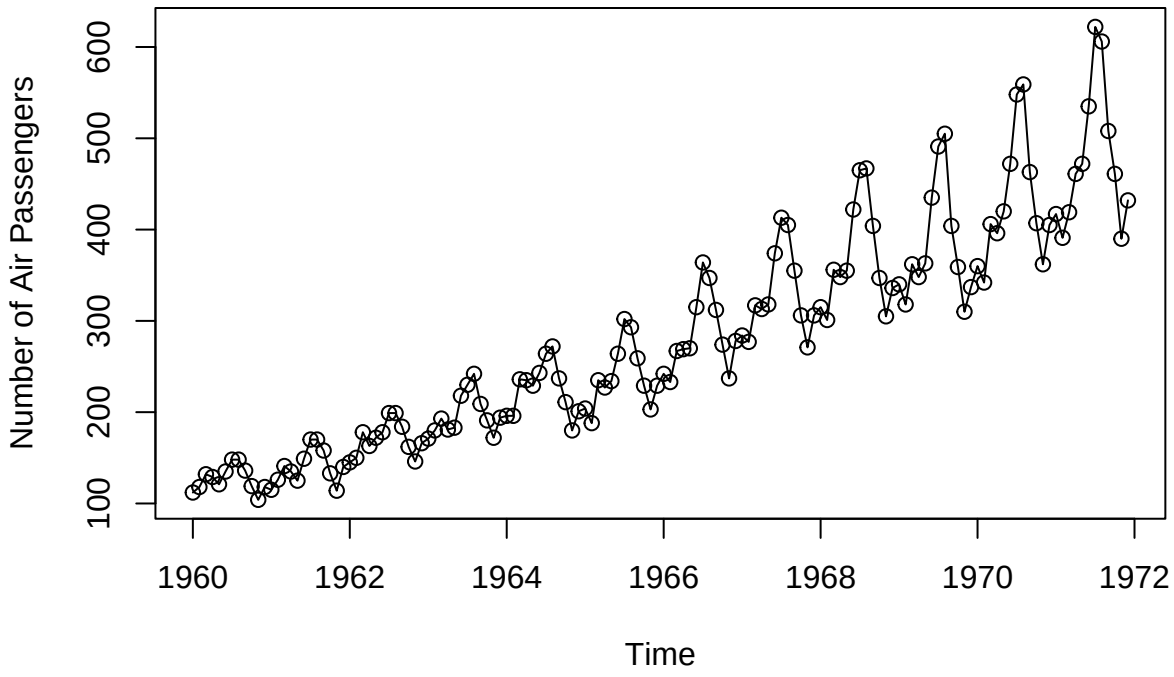
It is evident that this deterministic trend model fails to account for the autocorrelation present in the time series. A seasonal ARIMA model, as demonstrated in Chapter 10, provides a significantly better fit for these data.

Exercise 10.9

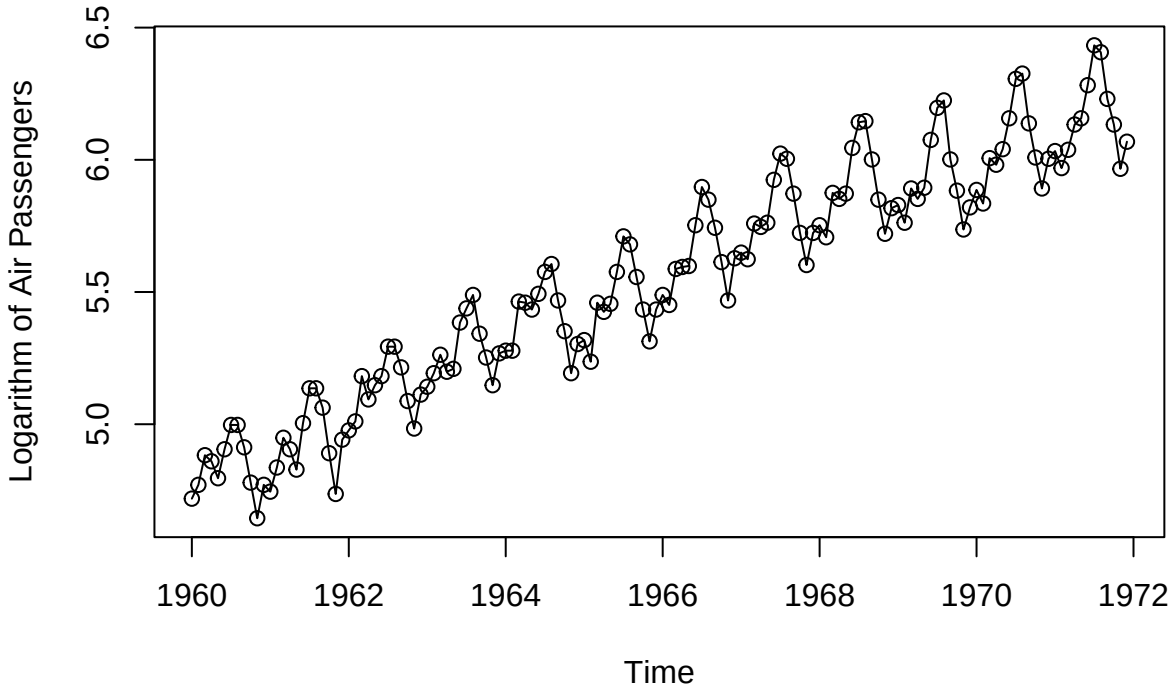
a

```
data(airpass)
passenger_data = airpass
log_passenger_data = log(airpass)

plot(passenger_data, type='o', ylab='Number of Air Passengers')
```



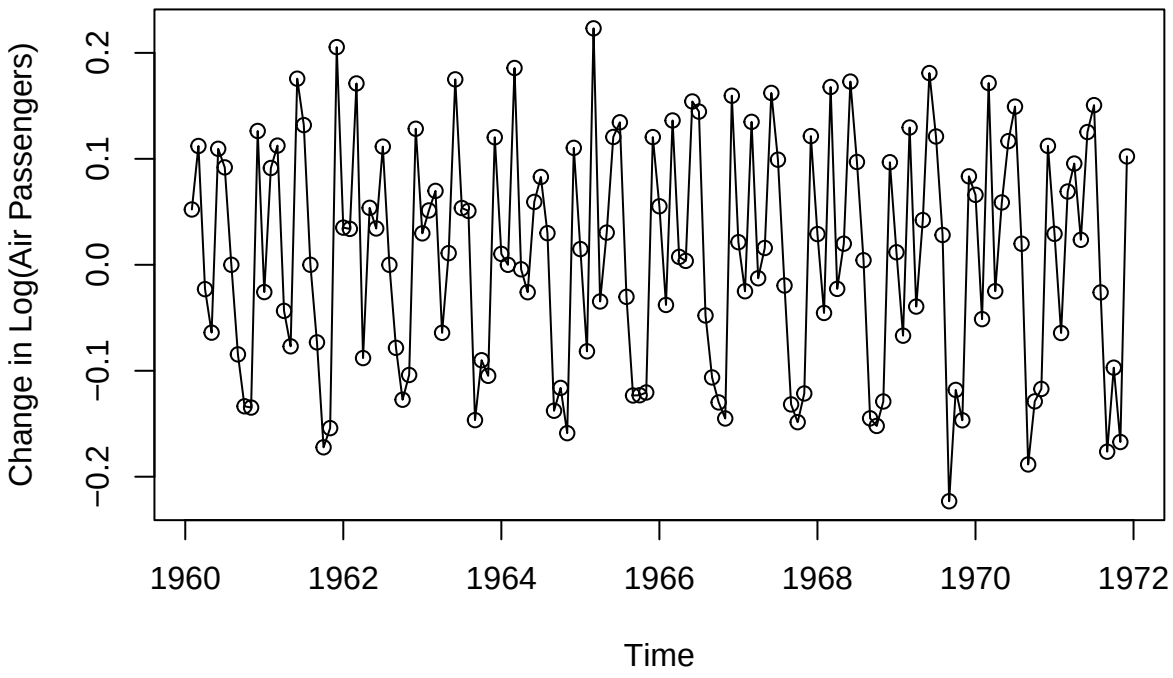

```
plot(log_passenger_data, type='o', ylab='Logarithm of Air Passengers')
```



In the original series, the seasonal fluctuations grow in amplitude over time, reflecting a non-constant variance. After applying a logarithmic transformation, these fluctuations appear more consistent, suggesting that the transformation has effectively stabilized the variance.

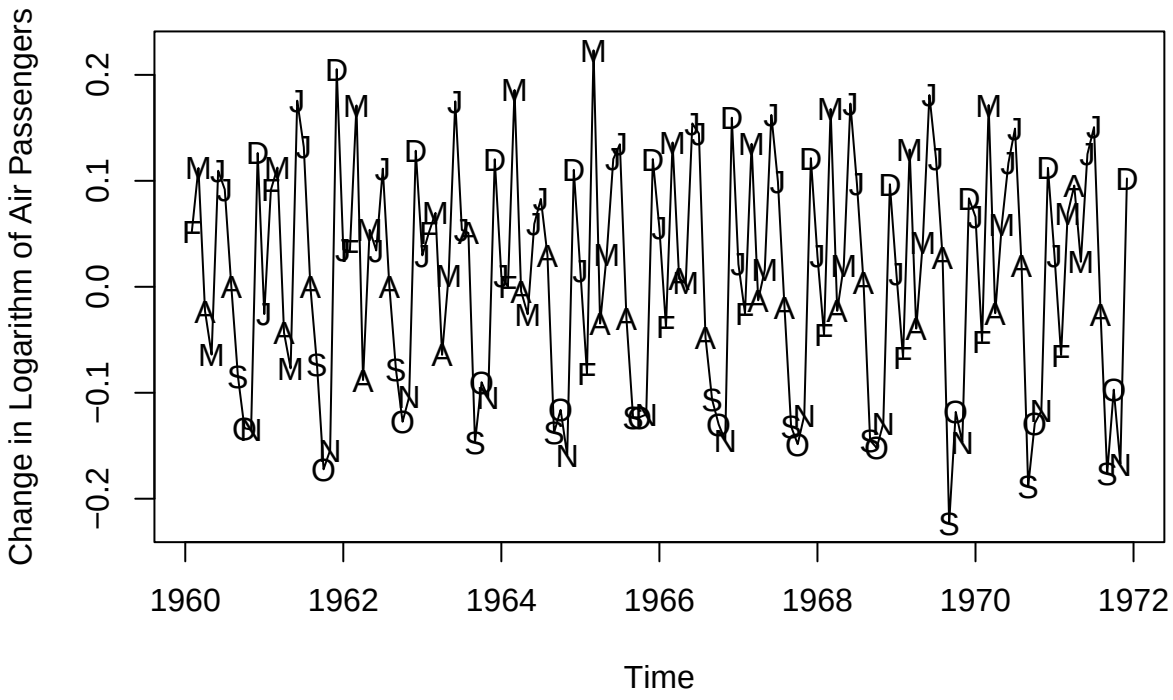
b

```
diff_passenger_data = diff(log_passenger_data)
plot(diff_passenger_data, type='o', ylab='Change in Log(Air Passengers)')
```



Differencing the logged airline passenger series removes the trend, making the series nearly stationary. However, the seasonal pattern persists, indicating periodic variations in the rate of passenger change. This transformation is essential for stabilizing variance and preparing the data for time series modeling, such as ARIMA.

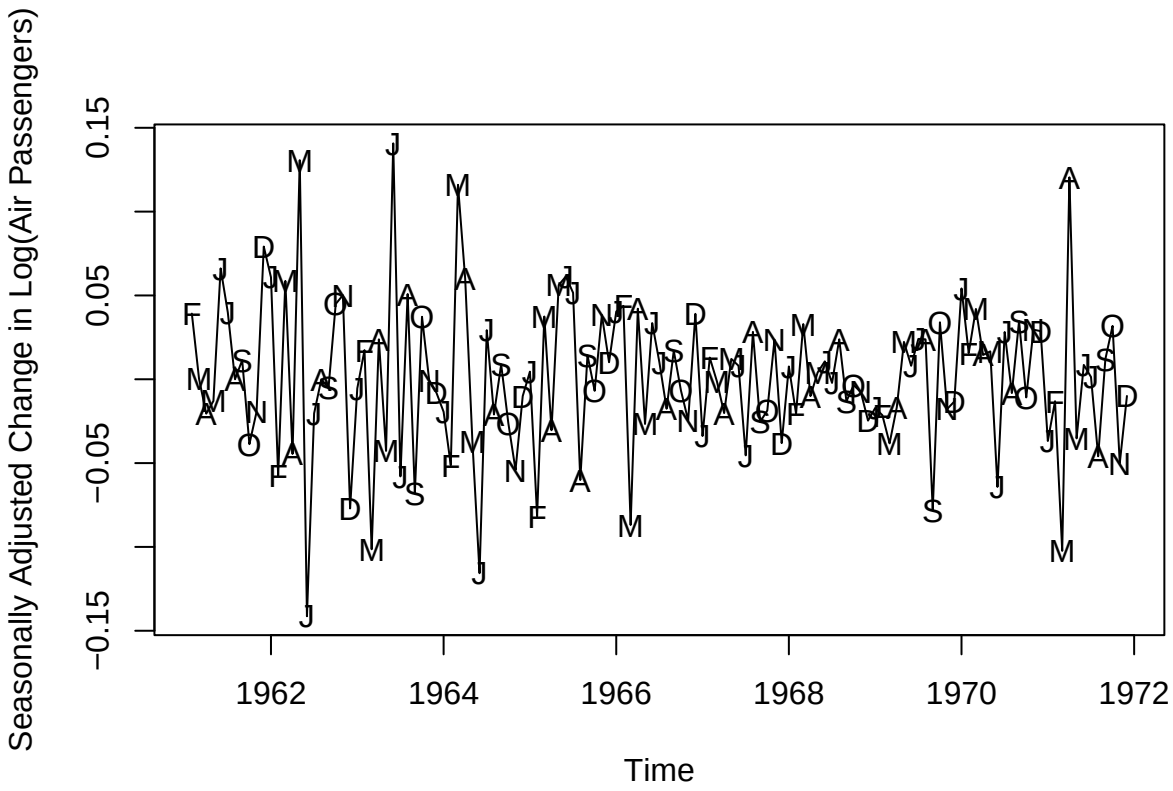
```
plot(diff_passenger_data, type='l', ylab='Change in Logarithm of Air Passengers')
points(diff_passenger_data, x=time(diff_passenger_data), pch=as.vector(season(diff_passenger_data)))
```



The plot of the first-differenced logarithmic data, with points labeled by their corresponding months, clearly reveals seasonal patterns. These repeated fluctuations highlight the significance of seasonality. By removing the trend and stabilizing variance, the transformation makes the series suitable for seasonal time series models like SARIMA.

c

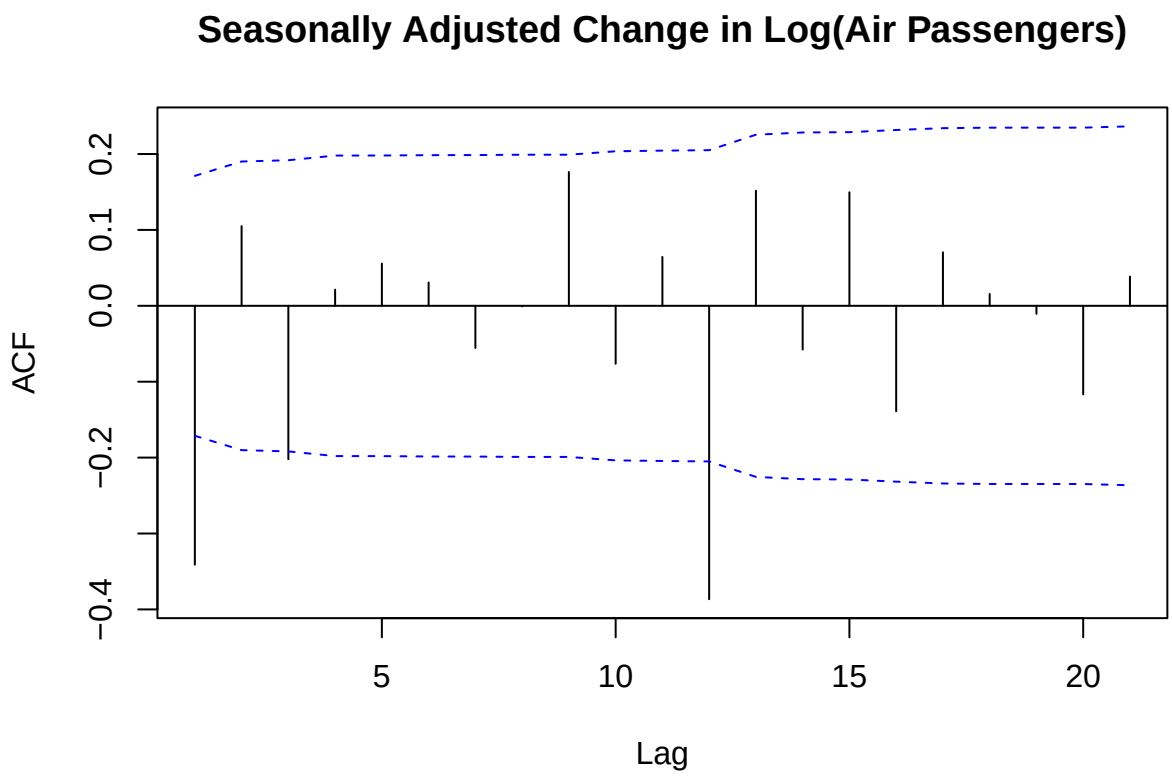
```
seasonal_diff_passenger_data = diff(diff_passenger_data, lag=12)
plot(seasonal_diff_passenger_data, type='l', ylab='Seasonally Adjusted Change in Log(Air Passengers)')
points(seasonal_diff_passenger_data, x=time(seasonal_diff_passenger_data),
       pch=as.vector(season(seasonal_diff_passenger_data)))
```



The plot of the seasonal difference of the first-differenced logged series shows that both seasonal and non-seasonal stationarity have been achieved. Applying a first difference and seasonal differencing (with a lag of 12) removes the trend and seasonality, leaving a stationary series. The remaining variability is largely random noise, making the series appropriate for SARIMA modeling.

d

```
acf(as.vector(seasonal_diff_passenger_data), ci.type='ma', main='Seasonally Adjusted Change in Log(Air
```



The ACF plot for the seasonal difference of the first-differenced logged series highlights significant autocorrelations at seasonal lags (e.g., 12, 24), indicating residual seasonality. The slower decay at other lags suggests non-seasonal patterns as well. This behavior reinforces the need for a SARIMA model to capture both seasonal and non-seasonal components.

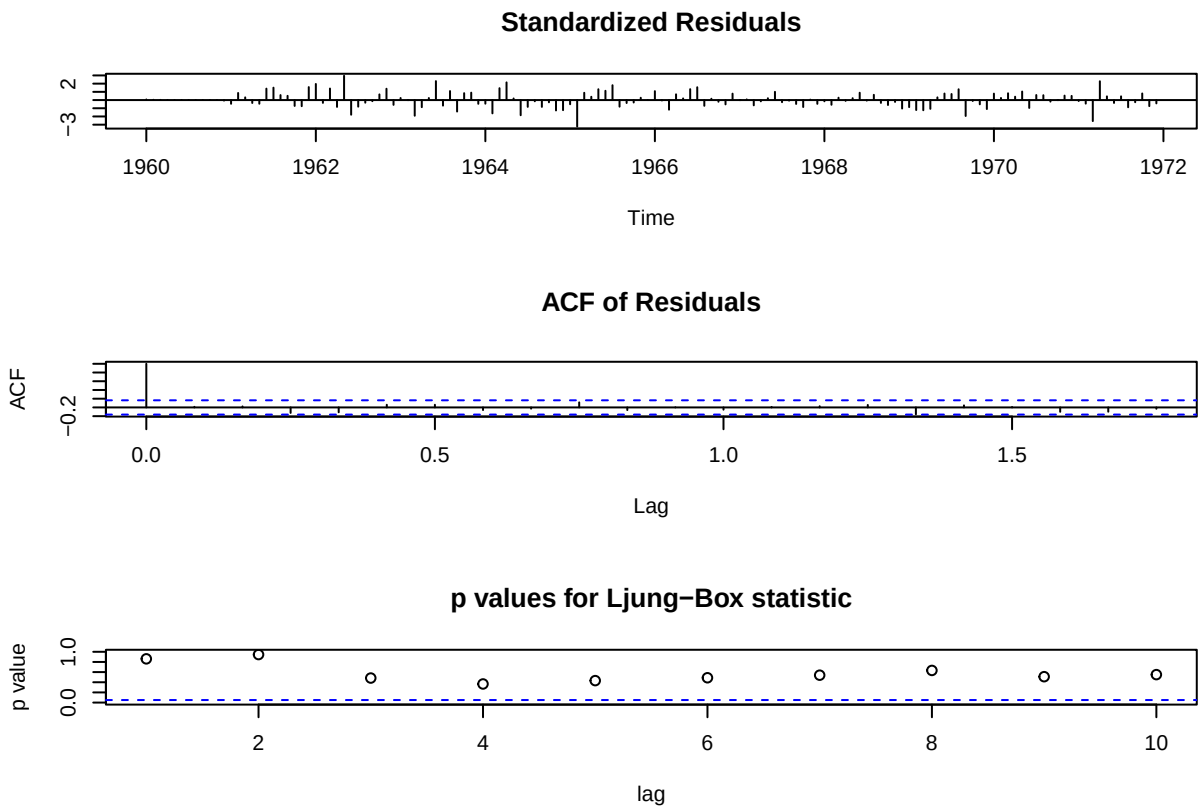
e

```
model_fit = arima(log_passenger_data, order=c(0,1,1), seasonal=list(order=c(0,1,1), period=12))
model_fit
```

```
##
## Call:
## arima(x = log_passenger_data, order = c(0, 1, 1), seasonal = list(order = c(0,
##     1, 1), period = 12))
##
## Coefficients:
##           ma1     sma1
##       -0.4018  -0.5569
## s.e.   0.0896   0.0731
##
## sigma^2 estimated as 0.001348:  log likelihood = 244.7,  aic = -485.4
```

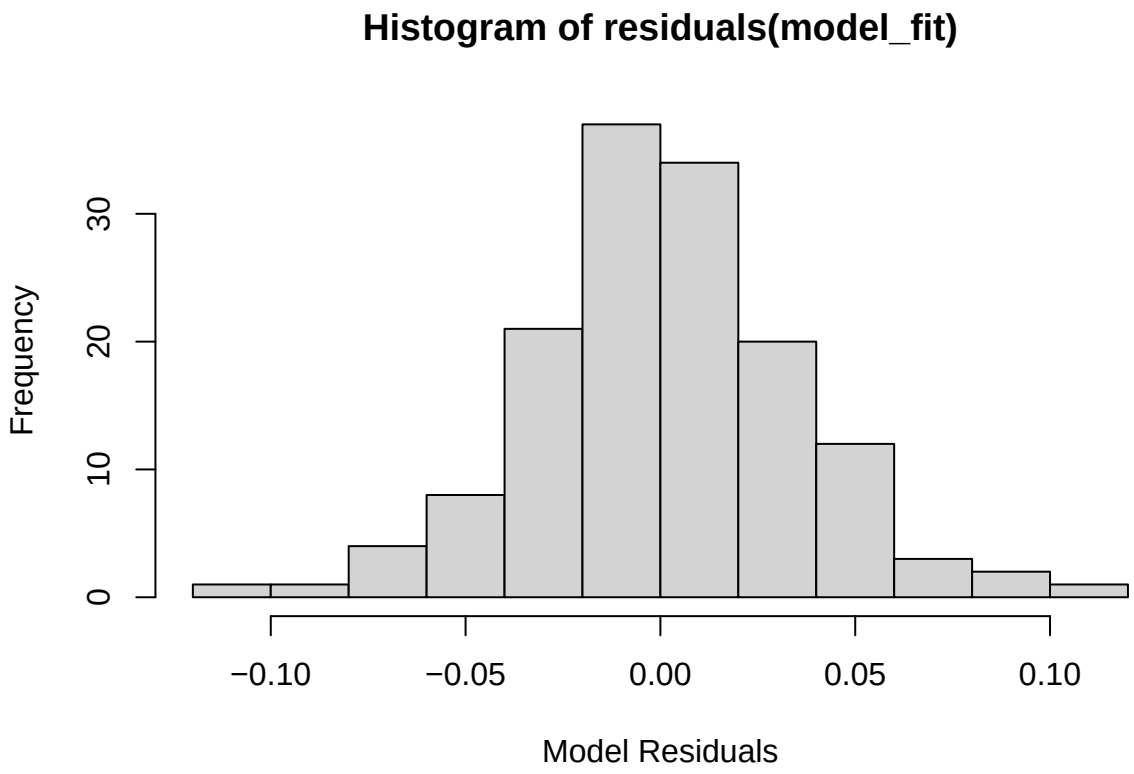
f

```
tsdiag(model_fit)
```

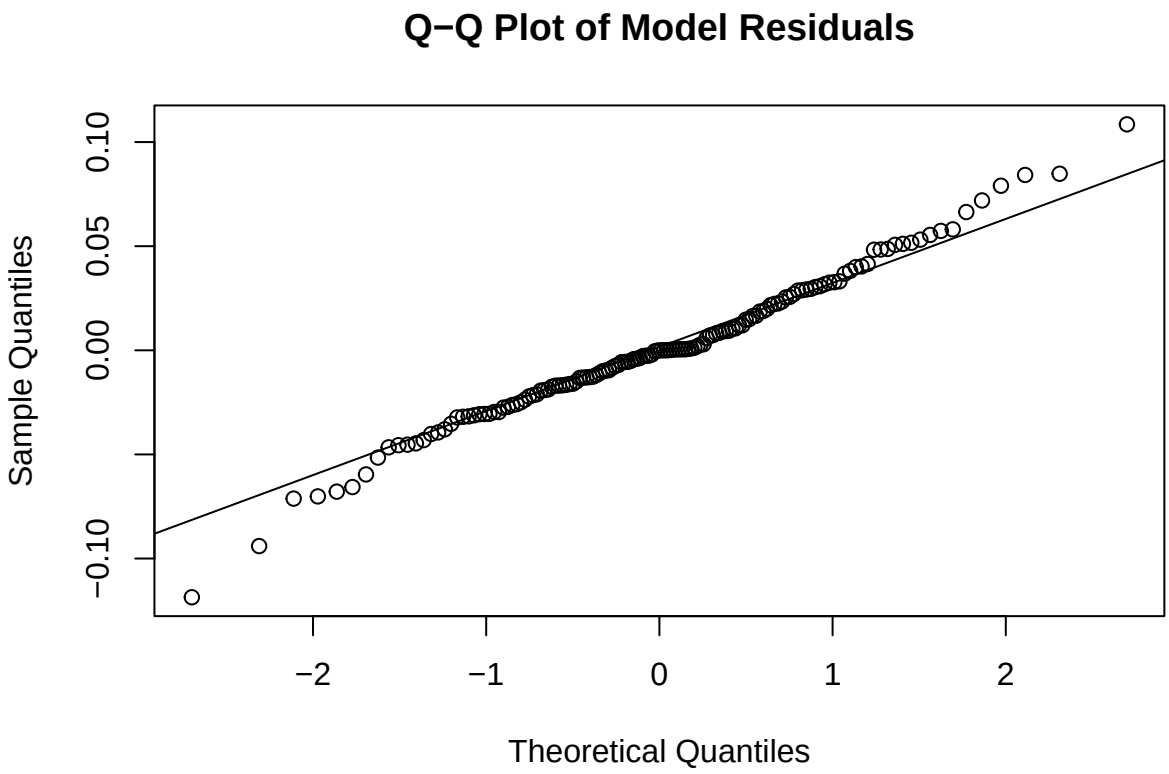


Normality of Residuals:

```
hist(residuals(model_fit), xlab='Model Residuals')
```




```
qqnorm(residuals(model_fit), main="Q-Q Plot of Model Residuals")
qqline(residuals(model_fit))
```



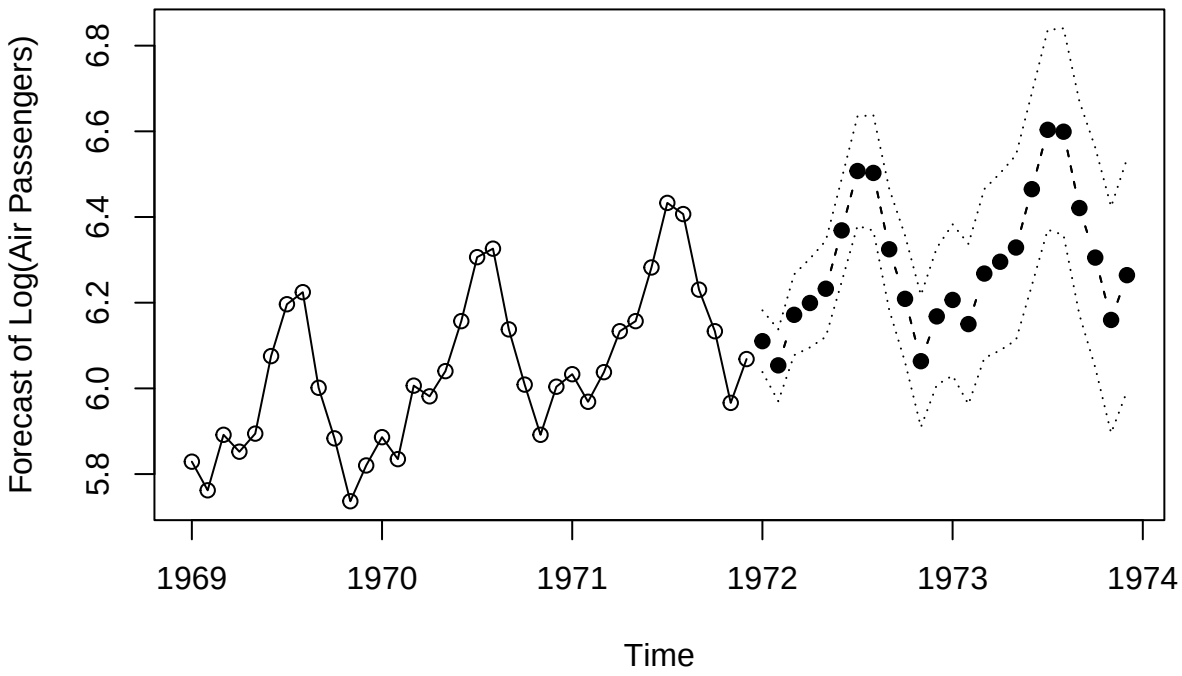

```
shapiro.test(residuals(model_fit))
```

```
##
##  Shapiro-Wilk normality test
##
## data:  residuals(model_fit)
## W = 0.98637, p-value = 0.1674
```

The Shapiro-Wilk test does not detect significant deviations from normality in the residuals, allowing the model to be confidently used for forecasting

g

```
plot(model_fit, n1=c(1969,1), n.ahead=24, pch=19, ylab='Forecast of Log(Air Passengers)')
```



The forecast plot of the log-transformed airline passenger series illustrates an upward trend with seasonal variations that align well with historical patterns. The confidence intervals widen as the forecast horizon increases, reflecting greater uncertainty over time.

9.4) Average monthly temperature May 1976

(a) Estimated cosine trend model is

$T(t) = 46.2660 + (-26.7070) \cdot \cos(2\pi t) - 2.1697 \cdot \sin(2\pi t)$

For May t = (1976 + 4/12) or (12 + 4/12)

= 1976.3333

$\cos(2\pi t) = -0.449$

$\sin(2\pi t) = 0.866130$

Substituting in (1)

$T(t) = 46.2660 + 26.7070 \times -0.999 - 2.1697 \times 0.866130$

$T(t) = 57.737°C$

(b) 95% prediction interval ≈ 1.96

$\sqrt{(ro)} = 3.719°F$

$57.7 \pm 1.96(3.719) \approx 57.7 \pm 7.4$

We are 95% confident that the May 1976 average temperature in Dubuque, Iower, "will" be between 50.3°F and 65.1°F.